

Séance 3 : Factory Pattern

3.1 Définition du Factory Pattern

Le **Factory Pattern** (Patron de Fabrique) est un pattern de création qui définit une interface pour créer un objet, mais laisse les sous-classes décider de la classe à instancier.

Problème résolu : Comment créer des objets sans spécifier leur classe exacte ?

3.2 Pourquoi utiliser un Factory ?

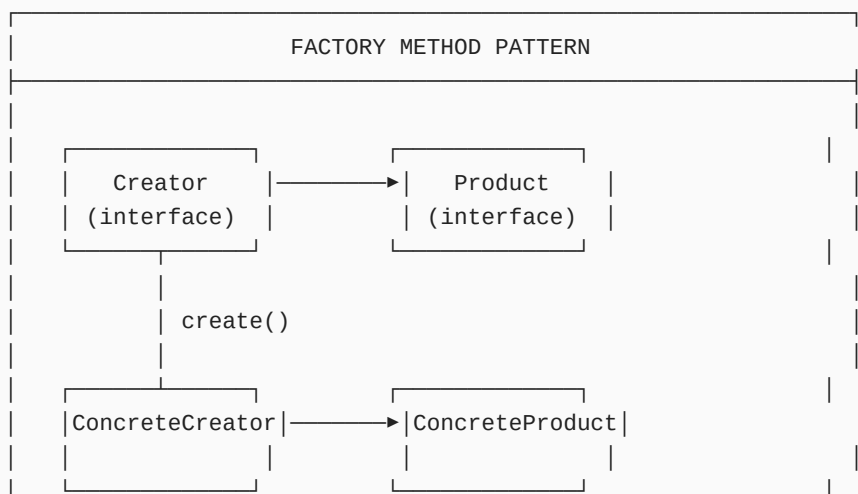
Sans Factory (code problématique) :

```
class NotificationService {
  send(message: string, type: string) {
    if (type === 'email') {
      const notification = new EmailNotification();
      notification.send(message);
    } else if (type === 'sms') {
      const notification = new SMSNotification();
      notification.send(message);
    } else if (type === 'push') {
      const notification = new PushNotification();
      notification.send(message);
    }
    // Ajouter un nouveau type = modifier cette méthode
  }
}
```

Problèmes :

1. **Violation du principe ouvert/fermé (OCP)** : Ajouter un type nécessite de modifier le code existant
2. **Couplage fort** : Le service connaît toutes les classes concrètes
3. **Code conditionnel** : Les chaînes if/else sont difficilement maintenables

3.3 Structure du Factory Pattern



3.4 Implémentation en TypeScript

Étape 1 : Définir l'interface du Produit

```
interface Notification {  
    send(message: string): void;  
}
```

Étape 2 : Implémenter les produits concrets

```
class EmailNotification implements Notification {  
    send(message: string): void {  
        console.log(`✉ Email sent: ${message}`);  
    }  
}  
  
class SMSNotification implements Notification {  
    send(message: string): void {  
        console.log(`📱 SMS sent: ${message}`);  
    }  
}  
  
class PushNotification implements Notification {  
    send(message: string): void {  
        console.log(`🔔 Push notification sent: ${message}`);  
    }  
}
```

Étape 3 : Créer l'interface du Créateur

```
abstract class NotificationFactory {  
    abstract createNotification(): Notification;  
  
    notify(message: string): void {  
        const notification = this.createNotification();  
        notification.send(message);  
    }  
}
```

Étape 4 : Implémenter les créateurs concrets

```
class EmailNotificationFactory extends NotificationFactory {  
    createNotification(): Notification {  
        return new EmailNotification();  
    }  
}  
  
class SMSNotificationFactory extends NotificationFactory {  
    createNotification(): Notification {  
        return new SMSNotification();  
    }  
}
```

```

class PushNotificationFactory extends NotificationFactory {
  createNotification(): Notification {
    return new PushNotification();
  }
}

```

Étape 5 : Utilisation

```

// Le client utilise la factory sans connaître la classe concrète
function sendNotification(factory: NotificationFactory, message: string) {
  factory.notify(message);
}

// Utilisation
sendNotification(new EmailNotificationFactory(), "Hello via Email!");
sendNotification(new SMSNotificationFactory(), "Hello via SMS!");
sendNotification(new PushNotificationFactory(), "Hello via Push!");

```

3.5 Variantes du Factory Pattern

Simple Factory (Factory Method simplifié)

```

class NotificationFactory {
  static create(type: string): Notification {
    switch (type) {
      case 'email': return new EmailNotification();
      case 'sms': return new SMSNotification();
      case 'push': return new PushNotification();
      default: throw new Error(`Unknown notification type: ${type}`);
    }
  }
}

// Utilisation
const notification = NotificationFactory.create('email');
notification.send('Hello!');

```

Abstract Factory

Pour créer des **familles** d'objets liés.

```

// Interface de la factory abstraite
interface UIFactory {
  createButton(): Button;
  createInput(): Input;
}

// Factory concrète pour Material Design
class MaterialUIFactory implements UIFactory {
  createButton(): Button {
    return new MaterialButton();
  }
  createInput(): Input {
    return new MaterialInput();
  }
}

```

```
// Factory concrète pour Bootstrap
class BootstrapUIFactory implements UIFactory {
    createButton(): Button {
        return new BootstrapButton();
    }
    createInput(): Input {
        return new BootstrapInput();
    }
}
```

3.6 Avantages et Inconvénients

✓ Avantages

- **Respecte l'OCP** : Ajouter un nouveau type ne modifie pas le code existant
- **Réduit le couplage** : Le client ne dépend que de l'interface
- **Centralise la création** : Un seul endroit pour gérer la création d'objets
- **Facilite les tests** : On peut injecter des mocks facilement

✗ Inconvénients

- **Complexité accrue** : Plus de classes à créer
- **Maintien** : Chaque nouveau type nécessite une nouvelle factory
- **Peut être overkill** : Pour des cas simples avec peu de variants

3.7 Cas d'utilisation

1. **Systèmes de notification** : Email, SMS, Push, Slack...
2. **Paielements** : CreditCard, PayPal, Bitcoin...
3. **Stockage** : LocalStorage, SessionStorage, IndexedDB...
4. **Connexion** : MySQL, PostgreSQL, MongoDB...
5. **Rendu** : PDF, HTML, Excel...

Exercice

Système de Paiement

Consigne : Implémentez un système de paiement utilisant le Factory Pattern.

```
// Types disponibles
type PaymentType = 'creditcard' | 'paypal' | 'crypto';

// Interface du produit
interface PaymentProcessor {
    pay(amount: number): void;
    refund(transactionId: string): void;
}

// TODO: Implémentez les classes concrètes
// - CreditCardPayment
// - PayPalPayment
// - CryptoPayment
```

```
// TODO: Créez la Factory
// - PaymentFactory avec une méthode create typée avec l'union PaymentType

// TODO: Testez votre implémentation
```

✓ Correction

Correction Exercice : Système de Paiement

```
// Types disponibles (union typée)
type PaymentType = 'creditcard' | 'paypal' | 'crypto';

// Interface du produit
interface PaymentProcessor {
    pay(amount: number): void;
    refund(transactionId: string): void;
}

// Produits concrets
class CreditCardPayment implements PaymentProcessor {
    pay(amount: number): void {
        console.log(`💳 Credit Card: Processing payment of ${amount}`);
        // Logique de paiement par carte
    }

    refund(transactionId: string): void {
        console.log(`💳 Credit Card: Refunding transaction ${transactionId}`);
    }
}

class PayPalPayment implements PaymentProcessor {
    pay(amount: number): void {
        console.log(`📄 PayPal: Processing payment of ${amount}`);
        // Logique PayPal
    }

    refund(transactionId: string): void {
        console.log(`📄 PayPal: Refunding transaction ${transactionId}`);
    }
}

class CryptoPayment implements PaymentProcessor {
    pay(amount: number): void {
        console.log(`₿ Crypto: Processing payment of ${amount}`);
        // Logique crypto
    }

    refund(transactionId: string): void {
        console.log(`₿ Crypto: Refunding transaction ${transactionId}`);
    }
}

// Factory avec typage strict
class PaymentFactory {
    // Mapping des types vers les classes concrètes
    private static readonly creators: Record<PaymentType, () => PaymentProcessor> = {
```

```

    creditcard: () => new CreditCardPayment(),
    paypal: () => new PayPalPayment(),
    crypto: () => new CryptoPayment(),
};

// La méthode create accepte uniquement les types de l'union
static create(type: PaymentType): PaymentProcessor {
    return this.creators[type]();
}
}

// Utilisation
const payment1 = PaymentFactory.create('creditcard');
payment1.pay(100);
payment1.refund('TX123');

const payment2 = PaymentFactory.create('paypal');
payment2.pay(50);

const payment3 = PaymentFactory.create('crypto');
payment3.pay(200);

// ❌ Erreur de compilation : Argument of type '"bitcoin"' is not assignable to parameter
of type 'PaymentType'
// const payment4 = PaymentFactory.create('bitcoin');

// ✅ Avantage : autocomplétion et vérification à la compilation
// Tapez PaymentFactory.create( et vous verrez les 3 options possibles

```



Résumé de la séance

- Le **Factory Pattern** sépare la création de l'utilisation
- Il existe 3 variantes : **Simple Factory**, **Factory Method**, **Abstract Factory**
- **Avantages** : Respecte l'OCP, réduit le couplage, facilite les tests
- **Inconvénients** : Complexité accrue, maintien des factories
- **Cas d'usage** : Systèmes de paiement, notifications, logs, documents...
- **TypeScript** : Utiliser des **unions de types** pour un typage strict des paramètres de la factory



Pour aller plus loin

- Explorer le **Abstract Factory** pour des familles d'objets
- Combiner Factory avec Singleton pour des factories globales
- Lire les exemples du GoF sur le Factory Method